ToggleMaster — Roteiro final de gravação

Tech Challenge Fase 2 · de monolito a microsserviços no Amazon EKS · duração-alvo: 18 a 19 minutos

Como usar este roteiro

Papel	Responsabilidade
Apresentador A (azul)	Deixas 1 a 18: abertura, arquitetura, Docker, ambiente local e AWS (≈ 9 min)
Apresentador B (laranja)	Deixas 19 a 37: Kubernetes, segredos, Ingress, fluxo funcional, HPA, DynamoDB e encerramento (≈ 10 min)
Operador de tela (cinza)	Troca a tela e roda o comando da deixa **antes** de o apresentador começar a fala. Os comandos estão no anexo, numerados igual às deixas.

Regras de sincronismo

- O operador muda a tela, espera o resultado aparecer e faz um sinal (ou diz "ok"). Só então o apresentador lê a fala.
- O apresentador lê **apenas a coluna da direita**. A coluna cinza é instrução para o operador.
- Comandos lentos (pods subindo, HPA escalando) devem ser cortados na edição. O tempo da tabela já considera esses cortes.
- Trechos **[opcional]** podem ser pulados se o tempo passar de 19 minutos.
- Gravar a Parte A e a Parte B separadamente e unir na edição.

Segurança na tela
Nunca mostrar: .env ou .env.aws, AWS Details, senhas, tokens, valores do Secrets Manager, a MASTER_KEY, API keys ou saídas de env/set. A MASTER_KEY deve estar numa variável carregada antes da gravação.

Checklist antes de gravar

#	Verificação	Comando / onde
1	Nove containers locais saudáveis	docker compose ps
2	kubectl apontando para o EKS correto	kubectl config current-context
3	Hop limit 2 nos nós atuais (eles podem ter sido recriados)	Anexo, item C1
4	Cinco Deployments disponíveis e três Jobs SQL concluídos	kubectl get deploy,jobs -A
5	Quatro ExternalSecrets sincronizados	kubectl get externalsecret -A
6	Load Balancer respondendo em /auth, /flags, /rules e /evaluate	Anexo, deixa 29
7	HPAs recebendo métricas (sem <unknown>)	kubectl get hpa -A
8	Carga ensaiada: saber quantos pods de carga levam a CPU acima de 30%	Anexo, deixa 31
9	jq disponível no terminal do fluxo funcional	jq --version
10	Payload do POST /rules testado com o README do targeting	Ensaio da deixa 29
11	Fonte do terminal aumentada e notificações ocultas	—
12	Slides prontos: título com os nomes, lições aprendidas, próximos passos e encerramento	—

PARTE A — Arquitetura, ambiente local e AWS · Apresentador A · ≈ 9 min

Nº	Tempo	OPERADOR — o que mostrar	APRESENTADOR — fala
1	0:00	**Slide de título** com o nome do projeto e os nomes dos cinco integrantes.	**Apresentador A** Olá! Nós somos o grupo do Tech Challenge da Fase 2, e os nomes dos integrantes estão na tela. Vamos apresentar a evolução do ToggleMaster: de uma aplicação monolítica para uma arquitetura de microsserviços executada no Amazon EKS.
2	0:25	**Diagrama de arquitetura** completo (docs/arquitetura_aws_togglemaster.png).	**Apresentador A** Na Fase 1, o ToggleMaster era um monolito numa EC2, conectado a um PostgreSQL no RDS. Nesta fase, a solução foi dividida em cinco microsserviços, cada um com uma responsabilidade.
3	0:50	No diagrama, **destacar os cinco serviços** (zoom ou cursor sobre cada um enquanto é citado).	**Apresentador A** O auth-service, em Go, cria e valida as chaves de API. O flag-service, em Python, mantém as feature flags. O targeting-service, também em Python, mantém as regras de segmentação. O evaluation-service, em Go, decide se a flag está ativa para cada usuário e usa Redis como cache. E o analytics-service consome os eventos e grava os dados de uso.
4	1:30	No diagrama, **apontar o caminho** evaluation → SQS → analytics → DynamoDB.	**Apresentador A** O caminho de analytics é assíncrono. O evaluation publica cada resultado numa fila SQS e responde ao cliente sem esperar a gravação. O analytics consome a fila no seu ritmo e grava no DynamoDB. Assim, uma lentidão na análise não atrasa a avaliação das flags.

Nº	Tempo	OPERADOR — o que mostrar	APRESENTADOR — fala
5	2:00	**VS Code**: Dockerfile do auth-service (Go). Depois, o do flag-service (Python).	**Apresentador A** Os cinco serviços têm Dockerfiles multi-stage. Nos serviços Go, o código é compilado no primeiro estágio e só o binário vai para uma imagem Alpine. Nos serviços Python, as dependências são instaladas no estágio de build e copiadas para a imagem final. Todos os containers rodam com um usuário sem privilégio de root.
6	2:40	**Terminal**: rodar o comando da deixa 6 (docker images).	**Apresentador A** Aqui vemos o resultado. As imagens Go ficam bem menores que as Python, porque levam apenas o binário, sem compilador nem interpretador. Imagem menor significa download mais rápido no cluster e menos superfície de ataque.
7	3:00	**Terminal**: rodar docker compose ps (deixa 7).	**Apresentador A** Antes de ir para a nuvem, validamos tudo localmente com o Docker Compose. São nove containers: os cinco serviços, dois PostgreSQL, um Redis e um LocalStack. No lugar do DynamoDB Local, escolhemos o LocalStack porque ele emula o DynamoDB e também o SQS, o que permite testar todo o fluxo assíncrono sem depender da AWS.
8	3:35	**Terminal**: rodar o laço de health checks (deixa 8).	**Apresentador A** Os cinco serviços respondem ao health check, nas portas 8001 a 8005.
9	3:50	**Relatório / manual técnico**, seção de problemas encontrados.	**Apresentador A** **[opcional]** Durante a implementação, corrigimos erros de compilação no código fornecido, como imports sem uso em Go, ajustamos o cliente Redis para funcionar com o ElastiCache e tratamos diferenças de ambiente entre Windows, containers e AWS.

Nº	Tempo	OPERADOR — o que mostrar	APRESENTADOR — fala
10	4:15	**Console AWS → EKS**: página do cluster (status Active).	**Apresentador A** Agora a infraestrutura na AWS, criada no AWS Academy, na região us-east-1. O núcleo é o cluster Amazon EKS, que gerencia o plano de controle do Kubernetes.
11	4:40	**EKS → aba Compute**: Managed Node Group.	**Apresentador A** Os workloads rodam num Managed Node Group com instâncias t3.medium, usando a LabRole do Academy. Configuramos no mínimo um nó, dois desejados e no máximo quatro.
12	5:10	**ECR**: lista dos cinco repositórios.	**Apresentador A** As imagens ficam em repositórios privados no Amazon ECR, um por microsserviço. A versão atual do evaluation inclui a correção do cliente Redis para operar com TLS e cluster mode no ElastiCache.
13	5:40	**RDS**: lista com as três instâncias PostgreSQL.	**Apresentador A** Agora a persistência, com três tipos de armazenamento escolhidos pelo tipo de dado. Para dados relacionais, temos três instâncias PostgreSQL privadas no RDS, uma para autenticação, uma para flags e uma para targeting. É o padrão database-per-service: nenhum serviço acessa diretamente as tabelas de outro.
14	6:25	**ElastiCache**: cluster Redis.	**Apresentador A** O evaluation está no caminho crítico, chamado a cada requisição. Por isso ele usa o Redis no ElastiCache para guardar por 30 segundos os dados combinados da flag e da regra. Num cache hit, ele não precisa chamar os serviços de flags e de targeting, e a latência cai.

Nº	Tempo	OPERADOR — o que mostrar	APRESENTADOR — fala
15	7:00	**DynamoDB**: tabela ToggleMasterAnalytics.	**Apresentador A** Os eventos de analytics ficam no DynamoDB, na tabela ToggleMasterAnalytics. Cada avaliação vira um evento, então o volume é alto e variável. O modelo NoSQL se adapta a esse padrão de escrita e não sobrecarrega os bancos transacionais.
16	7:35	**SQS**: fila evaluation-analytics-queue.	**Apresentador A** Entre o evaluation e o analytics está a fila Standard evaluation-analytics-queue, que desacopla os dois serviços e absorve picos de carga. Como uma fila Standard pode entregar a mesma mensagem mais de uma vez, idempotência e fila de mensagens mortas são evoluções para produção.
17	8:05	**EC2 → Security Groups**: regras 5432 (RDS) e 6379 (ElastiCache).	**Apresentador A** Nenhum banco nem o cache é público. Os Security Groups liberam a porta 5432 do RDS e a 6379 do Redis somente para o cluster EKS.
18	8:30	**Voltar ao diagrama**.	**Apresentador A** Com as imagens e os serviços gerenciados prontos, vamos ver como os microsserviços rodam e são expostos dentro do cluster.

PARTE B — Kubernetes, escalabilidade e encerramento · Apresentador B · ≈ 10 min

Nº	Tempo	OPERADOR — o que mostrar	APRESENTADOR — fala
19	9:00	**VS Code**: pasta k8s-aws com o kustomization.yaml aberto.	**Apresentador B** Os manifestos ficam na pasta k8s-aws do repositório, organizados por serviço. O Kustomize reúne todos os recursos e aplica o conjunto de forma consistente.
20	9:20	**Terminal**: kubectl config current-context e kubectl get ns (deixa 20).	**Apresentador B** Estamos conectados ao cluster EKS. Cada microsserviço tem seu próprio namespace: auth, flag, targeting, evaluation e analytics. Eles se comunicam pelo DNS interno do cluster, por exemplo flag-service.flag.svc.cluster.local.
21	9:50	**VS Code**: Deployment do flag-service. Rolar mostrando probes, resources e securityContext.	**Apresentador B** Os Deployments definem probes de startup, readiness e liveness, para o Kubernetes saber quando o pod está pronto e reiniciá-lo se travar. Também definem requests e limits de CPU e memória, que o autoscaling usa como referência. E aplicam controles de segurança: o container roda sem root, sem elevação de privilégio, sem capabilities e com o sistema de arquivos somente leitura.
22	10:40	**Terminal**: kubectl get externalsecret -A (deixa 22). Mostrar só nomes e status.	**Apresentador B** Nenhuma credencial fica nos YAMLs. O AWS Secrets Manager guarda as URLs dos bancos, as chaves internas e o token do Redis, e o External Secrets Operator sincroniza esses valores para Secrets do Kubernetes. Os componentes acessam a AWS pela LabRole dos nós, sem chaves fixas nos manifestos.

Nº	Tempo	OPERADOR — o que mostrar	APRESENTADOR — fala
23	11:20	**Terminal**: kubectl get jobs -A (deixa 23).	**Apresentador B** Três Jobs executam os scripts SQL de inicialização nos bancos de auth, flag e targeting. A criação das tabelas fica reproduzível e deixa de ser um passo manual.
24	11:45	**Terminal**: kubectl get pods,svc -A filtrado (deixa 24).	**Apresentador B** Os cinco serviços estão rodando, e cada um tem um Service do tipo ClusterIP, acessível apenas dentro do cluster.
25	12:05	**Terminal**: kubectl get ingress -A (deixa 25).	**Apresentador B** Para o acesso externo, as rotas /auth, /flags, /rules e /evaluate passam pelo mesmo Load Balancer e pelo Ingress Controller. O analytics não tem rota pública, porque é um worker interno.
26	12:30	**Slide**: evolução do Ingress (opcional) ou manter a tela anterior.	**Apresentador B** **[opcional]** Usamos o ingress-nginx, que segue funcionando, mas foi aposentado pelo projeto Kubernetes em março de 2026 e não recebe mais correções. Em produção, planejaríamos a migração para Gateway API ou para outro controller mantido.
27	12:55	**Terminal**: rodar a preparação do fluxo (deixa 27). Nada de chave aparece na tela.	**Apresentador B** Agora vamos testar o fluxo completo pelo Load Balancer. Primeiro, criamos uma chave de API no auth-service.
28	13:10	**Terminal**: criar a flag e a regra de 100% (deixa 28).	**Apresentador B** Com a chave, cadastramos a flag enable-new-dashboard no flag-service e uma regra de targeting que libera a flag para 100% dos usuários.

Nº	Tempo	OPERADOR — o que mostrar	APRESENTADOR — fala
29	13:30	**Terminal**: chamar o /evaluate (deixa 29). Destacar **result: true**.	**Apresentador B** Por fim, consultamos o evaluation para o usuário user-123. A resposta é true: o fluxo passou pelo Ingress, pelo auth, pelo flag, pelo targeting e pelo cache, e o evento já foi publicado na fila.
30	14:00	**Terminal**: kubectl top nodes e kubectl get hpa -A (deixa 30).	**Apresentador B** Agora a escalabilidade. O Metrics Server coleta o uso de CPU e memória de nós e pods, e essas métricas alimentam o Horizontal Pod Autoscaler. Temos HPAs para o evaluation e o analytics, entre uma e cinco réplicas, com alvo de 30% da CPU solicitada no Deployment.
31	14:35	**Tela dividida em três terminais.** Terminal 1: iniciar a carga (deixa 31). Terminais 2 e 3: HPA e pods em modo watch.	**Apresentador B** Vamos gerar chamadas contínuas ao evaluation, simulando vários clientes consultando a mesma flag. À direita, acompanhamos o HPA e os pods.
32	15:00	**Manter a tela** até a CPU passar do alvo e surgirem novos pods. Cortar a espera na edição.	**Apresentador B** A utilização de CPU passou do alvo. O HPA aumentou o número de réplicas, e novos pods do evaluation foram criados, sem nenhuma intervenção manual. O Service continua com um único endereço e distribui as requisições entre eles.
33	15:45	**Terminal 1**: encerrar a carga (deixa 33). Manter o watch até as réplicas caírem. Cortar a espera.	**Apresentador B** Ao encerrar a carga, o HPA respeita uma janela de estabilização e reduz as réplicas, liberando capacidade no cluster. Vale lembrar que o HPA ajusta pods, não instâncias: para reduzir o custo dos nós, seria preciso também autoscaling da infraestrutura. O analytics segue o mesmo critério quando o volume da fila aumenta o processamento.

Nº	Tempo	OPERADOR — o que mostrar	APRESENTADOR — fala
34	16:30	**Console → DynamoDB**: itens da tabela ToggleMasterAnalytics (ou o comando da deixa 34).	**Apresentador B** E aqui a prova ponta a ponta: cada avaliação publicou um evento no SQS, o analytics consumiu e gravou um item no DynamoDB. Estes registros foram gerados durante o teste.
35	17:05	**Slide**: lições aprendidas.	**Apresentador B** Nossas principais lições: validar localmente antes de ir para a nuvem; manter serviços e bancos separados; deixar segredos fora dos manifestos; sempre conferir o contexto do kubectl antes de aplicar qualquer coisa; e, no EKS, ajustar o hop limit dos nós para 2, sem o qual os pods não conseguiam usar a LabRole.
36	17:50	**Slide**: próximos passos.	**Apresentador B** **[opcional]** Como próximos passos, propomos pipeline de CI/CD, HTTPS, observabilidade, EKS Pod Identity ou IRSA, fila de mensagens mortas e idempotência, Gateway API e autoscaling também dos nós.
37	18:20	**Slide final** com os nomes do grupo.	**Apresentador B** Obrigado! Este foi o ToggleMaster na Fase 2: uma plataforma de microsserviços conteinerizada, implantada no Amazon EKS e integrada a serviços gerenciados da AWS.

Anexo — Comandos do operador (numerados pelas deixas)

Testar todos antes da gravação e deixar num arquivo de texto para copiar e colar.

C1 · Antes de gravar: hop limit 2 nos nós atuais (trocar NOME_DO_CLUSTER)

```
IDS=$(aws ec2 describe-instances --filters Name=tag:eks:cluster-name,Values=NOME_DO_CLUSTER \
  Name=instance-state-name,Values=running --query "Reservations[].Instances[].InstanceId" --output text)
for i in $IDS; do aws ec2 modify-instance-metadata-options --instance-id $i --http-put-response-hop-limit 2 \
  --http-endpoint enabled --query "InstanceMetadataOptions.HttpPutResponseHopLimit" --output text; done
```

Deixas 6 a 8 · Ambiente local

```
docker images | grep -E "auth-service|flag-service|targeting-service|evaluation-service|analytics-service"
docker compose ps
for p in 8001 8002 8003 8004 8005; do echo -n "$p: "; curl -s "localhost:$p/health"; echo; done
```

Deixas 20 a 25 · Inspeção do cluster

```
kubectl config current-context
kubectl get ns
kubectl get externalsecret -A
kubectl get jobs -A
kubectl get pods,svc -A | grep -E "auth|flag|targeting|evaluation|analytics"
kubectl get ingress -A
```

Antes da deixa 27 · Preparação (fora da gravação)

```
export MASTER_KEY='***carregar sem exibir***'
LB=$(kubectl get ingress -A -o jsonpath='{.items[0].status.loadBalancer.ingress[0].hostname}')
FLAG="enable-new-dashboard"
```

Deixa 27 · Criar a chave de API (não imprime a chave)

```
KEY=$(curl -s -X POST "http://$LB/auth/admin/keys" -H "Authorization: Bearer $MASTER_KEY" \
  -H "Content-Type: application/json" -d '{"name":"demo-video"}' | jq -r '.key')
[ -n "$KEY" ] && echo "chave criada" || echo "falhou"
```

Deixa 28 · Criar a flag e a regra de 100%

```
curl -s -X POST "http://$LB/flags" -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \
  -d "{\"name\":\"$FLAG\",\"description\":\"Demonstracao do video\",\"is_enabled\":true}"; echo
curl -s -X POST "http://$LB/rules" -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \
  -d "{\"flag_name\":\"$FLAG\",\"is_enabled\":true,\"rules\":{\"type\":\"PERCENTAGE\",\"value\":100}}"; echo
```

Deixa 29 · Avaliar a flag

```
curl -s "http://$LB/evaluate?user_id=user-123&flag_name=$FLAG"; echo
```

Deixa 30 · Métricas e HPA

```
kubectl top nodes
kubectl get hpa -A
```

Deixa 31 · Carga (Terminal 1) e acompanhamento (Terminais 2 e 3)

```
# Terminal 1 - se a CPU nao passar de 30% no ensaio, trocar 3 por 5
for i in 1 2 3; do
  kubectl run "carga-$i" --image=busybox:1.36 --restart=Never -- /bin/sh -c \
  'while true; do wget -q -O- "http://evaluation-service.evaluation.svc.cluster.local:8004/evaluate?user_id=user-123&flag_name=enable-new-dashboard" >/dev/null; done'
done
# Terminal 2
kubectl get hpa -n evaluation -w
# Terminal 3
kubectl get pods -n evaluation -w
```

Deixa 33 · Encerrar a carga

```
kubectl delete pod carga-1 carga-2 carga-3 carga-4 carga-5 --ignore-not-found
```

Deixa 34 · Prova no DynamoDB (alternativa ao console)

```
aws dynamodb scan --region us-east-1 --table-name ToggleMasterAnalytics --max-items 5 --output table
```

Dicas de edição

- Unir a Parte A e a Parte B e conferir a duração final (limite de 20 minutos).
- Cortar as esperas de criação de pods e de estabilização do HPA, mantendo o antes, o durante e o depois.
- Se passar de 19 minutos, remover primeiro as deixas marcadas como [opcional] (9, 26 e 36).